

SUPERBOARD

Platform Audit Report

Comprehensive White-Box Audit

1. Functional Testing
2. Security & Penetration Testing
3. Performance & Load Analysis
4. UX/UI & Accessibility (WCAG 2.1)
5. Code Quality & Architecture

Audit Type:	White-Box (Full Source Code Access)
Date:	August 05, 2026
Platform:	Superboard K-12 AI Tutoring Whiteboard
Tech Stack:	Next.js 16 / React 19 / Prisma / Supabase / Tailwind CSS 4
Auditor:	Automated White-Box Audit Engine

Table of Contents

Executive Summary

Audit Methodology

Section 1: Functional Testing

- 1.1 Feature Status Matrix
- 1.2 API Route Functionality
- 1.3 Button & Workflow Testing

Section 2: Security Audit

- 2.1 Critical Vulnerabilities
- 2.2 Authentication & Authorization
- 2.3 Data Exposure Risks
- 2.4 Dependency Vulnerabilities

Section 3: Performance & Load Testing

- 3.1 Bundle Size Analysis
- 3.2 TypeScript Error Inventory
- 3.3 Database Performance
- 3.4 Caching & Optimization

Section 4: UX/UI & Accessibility Review

- 4.1 WCAG 2.1 Compliance Matrix
- 4.2 Keyboard & Screen Reader Access
- 4.3 Contrast & Visual Design

Section 5: Code Quality & Architecture

- 5.1 Architecture Assessment
- 5.2 Technical Debt Inventory
- 5.3 Dead Code & Unused Dependencies

Consolidated Remediation Roadmap

Appendix A: Full Vulnerability Index

Appendix B: File Inventory

Executive Summary

This report presents the findings of a comprehensive white-box audit of the Superboard platform, a K-12 AI-powered tutoring whiteboard application. The audit was conducted with full access to the source code, database schema, server configuration, and API routes. Five distinct audit categories were examined: functional testing, security penetration testing, performance analysis, UX/UI accessibility compliance, and code quality assessment. The findings reveal that while the authentication framework and database layer are fundamentally sound, significant portions of the application remain in a scaffolded or non-functional state, with several critical security vulnerabilities requiring immediate remediation before any public launch.

Overall Risk Scores

Category	Score	Risk Level	Key Concern
Functional Testing	3/10	HIGH	~60% of features are non-functional stubs
Security	4/10	HIGH	2 CRITICAL auth gaps, no rate limiting
Performance	5/10	MEDIUM	No caching, 72 hidden TS errors, 25 npm vulns
Accessibility	3/10	HIGH	27 WCAG violations (14 Level A critical)
Code Quality	4/10	HIGH	1,518-line god file, secret leakage risk

The overall assessment indicates that Superboard is in a pre-alpha architectural stage. The core scaffolding (Supabase authentication, Prisma ORM, Stripe webhooks, agency management) demonstrates competent design patterns. However, the primary user-facing features (whiteboard drawing, AI integration, video conferencing, recording, PDF export, and GeoGebra integration) are all placeholder implementations that return mock data or log to console. From a security perspective, the absence of rate limiting across all endpoints and the completely unauthenticated LiveKit token generation endpoint represent the most urgent risks. The WCAG audit identified 27 accessibility violations, including missing landmark elements, absent keyboard navigation for the video panel, and insufficient color contrast ratios. Addressing these findings in priority order is essential before the platform can be considered ready for beta testing or public access.

Audit Methodology

This audit employed a **white-box methodology**, meaning the auditor had complete access to the application source code, database schema, server configuration files, environment variable handling patterns, and dependency manifests. This approach provides the deepest possible analysis compared to black-box testing, as it reveals vulnerabilities that may not be discoverable from the user interface alone, such as secret leakage patterns, improper authorization checks in API routes, and database query inefficiencies.

1.1 Scope

- **Source Code:** All 66+ files under `src/`, including pages, components, hooks, libraries, stores, types, and middleware. Also includes `prisma/schema.prisma`, `next.config.ts`, `tailwind.config.ts`, `Caddyfile`, and `package.json`.
- **API Routes:** All 17 API endpoints under `src/app/api/` were individually analyzed for authentication, authorization, input validation, error handling, and data exposure.
- **Components:** All 23 custom application components were audited for props validation, accessibility, memory leaks, XSS risks, state management, and responsive design.
- **Infrastructure:** Build configuration, deployment scripts, Hocuspocus real-time server, and reverse proxy configuration were reviewed.
- **Dependencies:** `npm audit` was run to identify known vulnerabilities. Bundle analysis was performed via Next.js build output.

1.2 Tools & Techniques

- **TypeScript Compiler:** `npx tsc --noEmit` was run to inventory all 72 TypeScript errors hidden by `ignoreBuildErrors: true`.
- **npm audit:** Automated vulnerability scanning identified 25 vulnerabilities (1 critical, 15 high, 7 moderate, 2 low).
- **Next.js Build Output:** Bundle analysis revealed 1.7 MB of client JavaScript across 33 chunks.
- **WCAG 2.1 Analysis:** Manual code review against all three WCAG levels (A, AA, AAA) for perceivable, operable, understandable, and robust criteria.
- **OWASP Patterns:** API routes were tested against OWASP Top 10 patterns including injection, broken authentication, sensitive data exposure, and missing rate limiting.

1. Section 1: Functional Testing

Functional testing evaluates whether every button, form, API call, and workflow performs as expected without throwing errors. This section maps every feature of the Superboard platform against its actual implementation status, revealing which features are fully operational, which are partially implemented, and which are entirely placeholder scaffolding. The analysis covers both user-facing interactions (clicking buttons, filling forms, navigating pages) and server-side workflows (API routes, database operations, Stripe webhook handling).

1.1 Feature Status Matrix

The following table summarizes the implementation status of every major feature in the Superboard platform. Features marked as "Non-Functional" have code that either returns hardcoded mock data, logs to console without performing real operations, or is entirely commented out. Features marked as "Partial" have some real logic but with significant gaps or bugs that prevent them from being used in production.

Feature	Status	Details
Whiteboard (Tldraw)	Non-Functional	Placeholder text displayed instead of canvas. All tool operations are no-ops.
Real-time (Yjs/Hocuspocus) Sync	Non-Functional	Server exists but all persistence hooks are commented out. No document save/load.
AI Integration (Claude)	Non-Functional	API route always returns placeholder JSON even when Anthropic API key is configured.
Video Conferencing (LiveKit)	Non-Functional	Mock JWT tokens generated. No real connection to LiveKit servers.
GeoGebra Integration	Non-Functional	Shows SVG placeholder. All plot commands logged to console only.
Session Recording	Non-Functional	Start/stop API endpoints do not exist. Button permanently disabled.
PDF Export	Non-Functional	All PDF generation code is commented out. No html2canvas or jsPDF installed.
Stripe Checkout	Partial	Webhook handler is wired, but /api/stripe/checkout route does not exist.
Focus Mode	Non-Functional	Entirely stub - broadcasts to console.log only.
All Toolkit Tools	Non-Functional	Math, Science, Language, General toolkits all use console.log stubs.
Auth (Supabase)	Functional	Login, signup, JWT verification all working correctly via Supabase.
Database (Prisma)	Functional	CRUD operations, relations, and schema are working correctly.
Tier Gating	Functional	Server-side tier checks on room creation, AI, and templates.

Feature	Status	Details
Usage Tracking	Functional	Credit deduction, period tracking, and usage dashboard all work.
Agency Invites	Functional	Create, accept, cancel invite flow works end-to-end.
Fingerprint Anti-Fraud	Functional	Collision detection and tier downgrade mechanism works.
Custom Domain Routing	Non-Functional	Middleware DB lookup entirely commented out.

The summary reveals a stark reality: only 6 out of 17 major features (35%) are fully functional. The remaining 65% are architectural scaffolding with TODO stubs, placeholder responses, or entirely commented-out implementations. This means that the core value proposition of Superboard as a tutoring whiteboard with AI, video, and real-time collaboration cannot currently be delivered to end users. The functional features (authentication, database, tier management, usage tracking, and agency invites) represent the administrative and billing infrastructure, not the teaching and learning experience.

1.2 API Route Functionality

All 17 API routes were tested for correct HTTP method handling, input processing, database interaction, and error response formatting. The table below summarizes the functional status of each route, including specific issues discovered during code review.

Route	Purpose	Status	Notes
GET /api/	Health Check	Functional	Returns static response. No issues.
POST /api/room	Create Room	Functional	Tier-gated, ownership verified. Subject not validated against enum.
GET /api/room	Get/List Rooms	Functional	Double-fetches room for tutor (redundant DB query).
GET/POST /api/room/participants	Join/List Participants	Functional	No authentication on either GET or POST.
GET/POST /api/room/templates	Template Management	Functional	Snapshot has no size validation. No template count limit.
POST /api/auth/register	User Registration	Functional	Legacy code allows primary key changes via email match.
GET /api/auth/profile	User Profile	Functional	Three redundant user lookups per request.
POST /api/ai/action	AI Proxy	Non-Functional	Always returns placeholder JSON. Credits are deducted for fake responses.
POST /api/stripe/webhook	Stripe Events	Functional	Proper signature verification. Metered billing uses wrong ID.
POST /api/livekit/token	Video Token	Non-Functional	No authentication. Returns mock JWT. LiveKit SDK API mismatch.
GET/POST /api/agency/invite	Agency Invites	Functional	8-char code with 10-attempt collision retry.

Route	Purpose	Status	Notes
GET/POST /api/agency/invite/[code]	Invite Accept	Functional	GET leaks invite status. POST requires email match.
POST /api/agency/invite/[code]/cancel	Cancel Invite	Functional	Proper ownership and status verification.
GET /api/agency/students	Student Count	Functional	Four sequential queries could be one aggregate.
DELETE /api/agency/subtutors/[id]	Remove Sub-Tutor	Functional	Resets tier to FREE with no cascade for their data.
GET /api/usage/current	Current Usage	Functional	Three redundant user lookups. No caching for high-frequency polling.
POST /api/usage/fingerprint	Anti-Fraud	Functional	Fingerprint hash format not validated. Auto-downgrade with no appeal.

1.3 Button & Workflow Testing

Every interactive element on the landing page, dashboard, room view, and settings panel was traced through the code to verify that click handlers are wired to real logic. The analysis identified several buttons that appear functional but perform no meaningful action when clicked, as well as critical navigation flows that are broken or lead to dead ends.

[HIGH] AI Credits Deducted for Placeholder Responses

The `/api/ai/action` route deducts AI credits from the user account on every successful request, but always returns a hardcoded placeholder JSON response regardless of whether the Anthropic API key is configured. This means users are charged credits for responses that contain no actual AI content. The `callTextAI` and `callVisionAI` functions in `lib/ai.ts` are dead code that is never called by the API route.

File: src/app/api/ai/action/route.ts:85-96

[HIGH] Stripe Checkout Route Does Not Exist

The `PaywallModal` component directs PRO upgrade clicks to `/api/stripe/checkout?plan=pro`, but this API route does not exist in the codebase. Clicking the upgrade button results in a 404 error. Only the webhook handler at `/api/stripe/webhook` is implemented, meaning there is no way for users to actually initiate a subscription payment.

File: src/components/premium/PaywallModal.tsx:89

[MEDIUM] Settings Gear Icon Has No Click Handler

The settings gear icon in the dashboard header renders a button element but the `onClick` handler calls `setShowSettings`, which is not defined as state in the component. This results in a silent runtime error when clicked, and the settings panel never opens.

File: src/app/page.tsx

[MEDIUM] Record Button Permanently Disabled

The RecordButton component is rendered in the UI for PRO and AGENCY users but its start/stop endpoints (/api/room/recording/start and /api/room/recording/end) do not exist. The button appears interactive but does nothing when clicked.

File: src/components/video/RecordButton.tsx

[LOW] PipVideoPanel One-Way State Trap

The inCall state in PipVideoPanel is initialized to true but once set to false (via "Leave Call"), there is no mechanism to set it back to true. The video panel permanently disappears for the remainder of the session with no way to rejoin.

File: src/components/video/PipVideoPanel.tsx:96

2. Section 2: Security Audit & Penetration Testing

The security audit examined the application for vulnerabilities across the OWASP Top 10 categories, focusing on authentication bypasses, authorization flaws, injection risks, sensitive data exposure, and infrastructure-level weaknesses. The findings are presented in severity order, from critical issues that could lead to immediate exploitation to lower-severity concerns that represent defense-in-depth improvements. All API routes were analyzed for proper authentication requirements, input validation practices, and safe data handling.

2.1 Critical Vulnerabilities

[CRITICAL] No Authentication on LiveKit Token Endpoint

The POST `/api/livekit/token` endpoint has absolutely no authentication check. Any anonymous user who knows or guesses a room ID can obtain a LiveKit token and join any active video session. The endpoint accepts arbitrary `roomId`, `userId`, and `userName` parameters without verification. An attacker could enumerate room IDs, impersonate any user, and potentially eavesdrop on tutoring sessions. The token generated is a mock JWT with a fake signature, but if this endpoint is connected to a real LiveKit server without adding authentication, the consequences would be severe. Even with the mock token, the endpoint leaks room existence and user identity patterns.

File: `src/app/api/livekit/token/route.ts`

[CRITICAL] No Rate Limiting on Any Endpoint

Not a single API route in the entire application implements rate limiting. No rate-limit library is installed, no middleware-level throttle exists, and no custom rate-checking logic is present anywhere in the codebase. This means all endpoints are vulnerable to brute-force attacks, credential stuffing, and denial-of-service. Specific attack vectors include: enumerating valid emails via `/api/auth/register`, brute-forcing 8-character invite codes on `/api/agency/invite/[code]`, exhausting AI credits on behalf of users by replaying requests, and flooding the database with rapid polling on usage and room endpoints.

File: All API routes under `src/app/api/`

[CRITICAL] Secret Keys Exported from Client-Accessible Modules

Several library files export secret API keys as named exports. Specifically, `LIVEKIT_API_SECRET` is exported from `lib/livekit.ts`, `STRIPE_SECRET_KEY` and `STRIPE_WEBHOOK_SECRET` from `lib/stripe.ts`, and `MATHPIX_APP_KEY` from `lib/mathpix.ts`. While Next.js tree-shaking currently prevents these from reaching the client bundle, any future refactoring that imports these modules from a client component would leak secrets to the browser. This is a ticking time bomb that violates the principle of defense in depth. Secrets should be accessed via environment variables only within API route handlers.

File: `src/lib/livekit.ts`, `src/lib/stripe.ts`, `src/lib/mathpix.ts`

2.2 Authentication & Authorization

[HIGH] Room Participants Endpoint Has No Authentication

Both GET and POST requests to `/api/room/participants` work without any authentication. Anyone can list all participants in any room (information disclosure) and anyone can join any active room as any identity. The POST handler accepts arbitrary `studentIdentity` and `studentName` values with no format validation or length limit, allowing an attacker to spam fake participants or store arbitrary data in the database. For agency billing which counts unique participants, this could be exploited to inflate billing charges.

File: `src/app/api/room/participants/route.ts`

[HIGH] Answer Key Security is Client-Side Only

The `AnswerKeyModal` component gates visibility using `if (!isTutor)` return null, which is a client-side check. A student could open browser DevTools, modify the Zustand store to set `isTutor: true`, and view all answer keys for any quiz or worksheet. The modal also displays a misleading "Encrypted" footer claim, when there is no actual encryption implemented. Answer keys should be fetched from a server-side endpoint that verifies tutor status before returning data.

File: `src/components/ai/AnswerKeyModal.tsx`

[HIGH] Hocuspocus Server Accepts All Connections

The Hocuspocus real-time collaboration server at `mini-services/hocuspocus-server/index.ts` has an `onAuthenticate` hook that accepts any connection without verifying room access. The comment indicates this is intentional for development, but if deployed to production, any user could connect to any room and modify the whiteboard document. All persistence hooks are also commented out, meaning documents exist only in memory and are lost on server restart.

File: `mini-services/hocuspocus-server/index.ts`

[MEDIUM] Invite Information Leakage

The GET `/api/agency/invite/[code]` endpoint leaks internal state to unauthenticated users. The error response includes detail: `Status: ACCEPTED` or `Status: CANCELLED`, which reveals whether an invite was sent and its outcome. Additionally, accessing an expired invite triggers an automatic status update to `EXPIRED`, meaning a scanner probing random codes can trigger mass status changes. The GET response also exposes the agency name, branding information, and the invited email address to anyone with the code.

File: `src/app/api/agency/invite/[code]/route.ts`

[MEDIUM] Room Details Leaked to Unauthenticated Users

The GET `/api/room` endpoint returns room details including the tutor name, tier, subject, and full board page snapshots to anyone who provides a valid room ID. No authentication is required to

read room content, which means board snapshots (potentially containing sensitive lesson material) are accessible without any access control.

File: src/app/api/room/route.ts:144-155

[MEDIUM] No CORS or Security Headers Configured

The `next.config.ts` file contains no security header configuration. There is no Content-Security-Policy, no X-Frame-Options, no X-Content-Type-Options, no Strict-Transport-Security, and no Referrer-Policy. The application relies entirely on Next.js defaults (same-origin only), which leaves it vulnerable to clickjacking (no X-Frame-Options: DENY) and MIME-type sniffing attacks.

File: next.config.ts

2.3 Data Exposure & Input Risks

[MEDIUM] No Input Validation Library Used (Zod Installed But Unused)

Zod is listed as a dependency (version ^4.0.2) but is never imported in any API route. All validation consists of manual `if (!field)` checks, which means there are no type-safe parsers, no maximum length checks on strings, no enum validation beyond manual comparisons, and no schema reuse between client and server. The prompt parameter to the AI action endpoint has no length limit, and `imageBase64` has no size limit, creating potential memory exhaustion vectors.

File: All API routes

[MEDIUM] KaTeX trust: true Enables HTML Injection

The KaTeX rendering configuration in `lib/katex.ts` sets `trust: true`, which allows LaTeX commands like `\html{...}` to render raw HTML. If user-controlled LaTeX content (e.g., from the AI response or student input) is rendered with this configuration, it creates a cross-site scripting (XSS) vector. Additionally, `strict: false` suppresses KaTeX error reporting, meaning malformed input fails silently rather than being rejected.

File: src/lib/katex.ts:23

[MEDIUM] Agency Logo URL Not Validated (Potential XSS)

The `BrandedHeader` and `WaitingRoom` components render agency-controlled `logoUrl` values directly in `img src` attributes. If an agency tutor sets their `logoUrl` to a `javascript: URI` or a `data URI` containing malicious content, it could execute in the user browser context. The URL should be validated to ensure it points to an allowed domain and uses an acceptable protocol (`https`).

File: src/components/branding/BrandedHeader.tsx:33, src/components/student/WaitingRoom.tsx:138

[MEDIUM] Primary Key Modification in Auth Register

The `/api/auth/register` route contains legacy code that allows changing a User record primary key (UUID) via Prisma update. If an existing user is found by email, the route updates both the id and

name fields. Changing a UUID primary key at runtime is a risky operation that could cause cascading foreign key failures and data inconsistency across related tables (Room, UsageLog, AgencyInvite). This should be removed and replaced with a proper database migration if needed.

File: src/app/api/auth/register/route.ts:60-63

2.4 Dependency Vulnerabilities

Running npm audit against the project dependencies revealed 25 vulnerabilities across the dependency tree. The following table summarizes the most significant findings by severity. Many of these are in transitive dependencies and can be resolved by running npm audit fix, though some require manual intervention due to breaking changes.

Severity	Package	Issue	Resolution
CRITICAL	effect (via @prisma/config)	AsyncLocalStorage context contamination under concurrent load	Update Prisma
HIGH	sharp@0.34.3	libvips CVEs (4 vulnerabilities)	Update to 0.35.3 (breaking)
HIGH	defu	Prototype pollution via __proto__	npm audit fix
HIGH	brace-expansion	Zero-step DoS via unbounded expansion	npm audit fix
HIGH	@babel/core	Arbitrary file read via sourceMappingURL	npm audit fix
MODERATE	uuid (via next-auth)	Missing buffer bounds check	Remove unused next-auth

3. Section 3: Performance & Load Analysis

Performance analysis examines how efficiently the application uses resources under both normal and high-traffic conditions. This section covers client-side bundle size, server-side build integrity, database query optimization, caching strategies, and dependency overhead. The analysis reveals several optimization opportunities that would significantly improve load times and reduce server costs, particularly around the complete absence of caching and the presence of 72 hidden TypeScript errors that mask potential runtime bugs.

3.1 Bundle Size Analysis

The Next.js build produces 1.7 MB of client JavaScript across 33 chunks. While tree-shaking successfully prevents unused dependencies (framer-motion, next-auth, next-intl) from reaching the client bundle, several optimizations remain. The largest single chunk is 244 KB (base64 polyfill), followed by 220 KB (Next.js runtime) and 154 KB (Next.js core). These are framework overhead and cannot be reduced, but the following table identifies reducible contributors.

Package	Bundle Size	Status	Notes
Stripe client lib	56 KB	Server-only	Should not be in any client-accessible chunk
@prisma/client	27 KB	Server-only	Prisma engine code must not reach client bundle
lucide-react	9.9 KB	Client	Acceptable (tree-shaked)
framer-motion	0 KB	Unused	Successfully tree-shaken but still in node_modules
next-auth	0 KB	Unused	Tree-shaken; also brings uuid with known CVE
next-intl	0 KB	Unused	Successfully tree-shaken

The Prisma client appearing in any client-accessible chunk is the most concerning finding. This can be resolved by adding serverExternalPackages: ["@prisma/client"] to next.config.ts. The Stripe library leakage should also be prevented by ensuring it is only imported within API route files that are explicitly server-only.

3.2 TypeScript Error Inventory

The next.config.ts file sets ignoreBuildErrors: true, which silently suppresses all TypeScript compilation errors during the build process. Running npx tsc --noEmit reveals 72 TypeScript errors across the codebase. These errors are not merely cosmetic; they indicate broken features, API mismatches with external libraries, and type safety violations that could lead to runtime errors. The following table categorizes these errors by type and impact.

Category	Count	Impact
Missing Names (TS2304)	~15	SectionLabel, LockOverlay, showSettings, tierColor, tierLabel not found
Wrong Properties (TS2339)	~15	Stripe: createUsageRecord removed, LiveKit: enableMicrophone gone
Type Mismatches (TS2322)	~10	TIER_LIMITS union type inconsistencies, Stripe API version mismatch
Duplicate Identifiers (TS2300)	2	useAppStore imported twice in QuizGenerator.tsx
External Code (TS2307)	4	socket.io-client imports, hocuspocus-server type errors
Other	~26	Nullable type issues, wrong function signatures, missing return types

The Stripe API version mismatch is particularly impactful. The code specifies `apiVersion 2025-06-30.basil` but the installed Stripe SDK expects `2026-07-29.dahlia`, causing type errors on `createUsageRecord` and subscription properties. The LiveKit SDK has also been updated with breaking changes (LiveKitClient export removed, `enableMicrophone/enableCamera` renamed), meaning the video integration code is not just non-functional but incompatible with the current SDK.

3.3 Database Performance

The Prisma schema contains zero explicit `@@index` declarations. While Prisma auto-creates indexes for foreign key fields defined by `@relation`, several common query patterns lack optimal indexes. Additionally, multiple API routes execute redundant database queries that could be consolidated. The following table identifies the most impactful missing indexes and the query patterns they would optimize.

Table	Missing Index	Query Pattern	Affected Route
RoomParticipant	roomId + lastActiveAt	WHERE roomId IN (...) AND lastActiveAt >= ...	agency/students
Room	tutorId + isActive	WHERE tutorId = X AND isActive = true (COUNT)	room/route.ts
AgencyInvite	invitedEmail	findFirst WHERE invitedEmail + status = PENDING	agency/invite
BoardPage	roomId + pageIndex	ORDER BY pageIndex ASC	room/route.ts
Template	tutorId + updatedAt	ORDER BY updatedAt DESC	room/templates
RoomParticipant	studentIdentity + lastActiveAt	Distinct student counting across periods	agency/students

3.4 Redundant Database Queries

Several API routes fetch the same user record multiple times within a single request. These redundant queries add unnecessary database load and increase response latency. The most egregious cases are listed below with specific line references and recommended consolidation strategies.

- **room/route.ts (lines 144 + 173):** Room is fetched twice with `findUnique`. The first fetch excludes tutor email for students; the second re-fetches including email for the tutor. This should be a single query with conditional include based on authentication status.
- **usage/current/route.ts (lines 30, 40, 52):** User record is looked up three times. Lines 30 and 52 query the same user when a user checks their own usage. These should be consolidated into a single query with all needed includes.
- **auth/profile/route.ts (lines 27, 38, 50):** Same three-lookup pattern. Agency check, sub-tutor parent check, and target user fetch all query the User table separately.
- **agency/students/route.ts (lines 18, 31, 39, 65):** Four sequential queries that could be replaced with a single aggregate SQL query using joins and group-by.

3.5 Caching & Optimization Gaps

The application has zero caching infrastructure. No `Cache-Control` headers are set on any API response. No route segment config (`revalidate`, `dynamic`, `fetchCache`) is exported from any route. No Next.js caching APIs (`unstable_cache`, `use cache`, `revalidatePath`, `revalidateTag`) are used anywhere. The `/api/usage/current` endpoint, which is polled every 30 seconds by the `UsageBar` component, hits the database on every single request. For a platform with concurrent users, this would generate thousands of unnecessary database queries per minute. Similarly, room data is re-fetched without any client-side or server-side caching, meaning navigating between dashboard and room view always triggers fresh database reads.

- **Recommendation:** Add `Cache-Control: s-maxage=10, stale-while-revalidate=30` to `/api/usage/current`.
- **Recommendation:** Use Next.js `unstable_cache` for room listing queries with a 60-second TTL.
- **Recommendation:** Replace raw `img` tags with `next/image` for automatic lazy loading, format conversion, and size optimization.
- **Recommendation:** Tighten `remotePatterns` from `hostname: "*"` to specific allowed domains.

4. Section 4: UX/UI & Accessibility Review

The accessibility audit evaluates the application against WCAG 2.1 standards across all four principles: Perceivable, Operable, Understandable, and Robust. The code-based review identified 27 violations: 14 at Level A (must fix for basic accessibility), 11 at Level AA (should fix for standard compliance), and 2 at Level AAA (nice to fix for enhanced usability). The most critical finding is that the video conferencing panel is entirely mouse-driven with zero keyboard support, effectively making the platform unusable for keyboard-only and screen reader users during video sessions.

4.1 WCAG 2.1 Compliance Matrix

ID	Criterion	Level	Violation	File Reference
P1	1.1.1	A	Decorative SVGs lack aria-hidden="true"	page.tsx:270-273
P2	1.1.1	A	Logo icons not hidden from assistive tech	page.tsx:247-249
P5	1.3.1	A	Color-only indicators not paired with text	page.tsx:832-833
P8	1.4.3	AA	text-gray-400 contrast ratio ~2.9:1 (fails 4.5:1)	page.tsx:451, 657
P10	1.4.11	AA	Low contrast icons on dark backgrounds	PipVideoPanel.tsx:319
O1	2.1.1	A	PiP video panel is entirely mouse-only	PipVideoPanel.tsx:120-195
O2	2.1.2	A	Keyboard trap in WaitingRoom overlay	WaitingRoom.tsx:130-189
O3	2.4.1	A	No skip-to-content link on any page	layout.tsx
O5	2.4.6	AA	Duplicate h1 / heading hierarchy issues	page.tsx:307, 845
O7	2.4.7	AA	Missing focus-visible on custom elements	page.tsx:952, PipVideoPanel
R1	1.3.1	A	No main landmark on landing page	page.tsx:242-760
R3	4.1.2	A	Clickable divs lack roles + keyboard support	page.tsx:952, 1146
R4	4.1.2	A	Video panel buttons lack aria-label	PipVideoPanel.tsx:436-509
R6	4.1.2	A	Duplicate nested toolbar roles	Whiteboard.tsx:170, Toolbar.tsx:83
U1	3.3.1	A	Error messages lack aria-live regions	page.tsx:713-721
U3	3.3.2	A	Labels without htmlFor associations	page.tsx:917, 931-933
U5	3.3.3	AA	No error recovery suggestions in auth forms	page.tsx:713-717
R10	4.1.3	AA	No aria-live for waiting room status	WaitingRoom.tsx:168-175
R11	4.1.3	AA	No live region for "Copied" confirmation	page.tsx:1331-1336
O9	2.5.5	AAA	Toolbar buttons 36x36px (below 44x44 AAA min)	Toolbar.tsx:89-97

4.2 Keyboard & Screen Reader Access

The most severe accessibility gap is the PiP (Picture-in-Picture) video panel, which relies entirely on mouse events for all interactions. The drag-to-move functionality uses only `onMouseDown` with no `onKeyDown` alternative. The resize handle is a `div` with only `onMouseDown`. The minimize/expand toggle is an `onClick` on a `div` element with no `tabIndex` or `role`. All video control buttons (camera, microphone, deafen, speaker output, and leave call) lack `aria-label` attributes, making them invisible to screen reader users. A keyboard-only user cannot move, resize, minimize, or interact with the video panel in any way.

The landing page lacks a skip-to-content link, which is a Level A requirement. Users who navigate via keyboard must tab through the entire navigation bar on every page load before reaching the main content. The landing page also lacks a main landmark element, wrapping all content in a plain `div` instead of semantic HTML. Multiple interactive elements throughout the dashboard (Quick Start items, board rows) are clickable `divs` with no `role="button"`, `tabIndex`, or `onKeyDown` handler, making them invisible to assistive technology and unreachable by keyboard.

4.3 Color Contrast & Visual Design

Several text elements fail the WCAG 2.1 Level AA contrast requirement of 4.5:1 for normal text. The most common offender is `text-gray-400` (`#9ca3af` on white), which achieves approximately 2.9:1 contrast ratio. This is used in multiple locations including the "and growing" text on the landing page, the "No credit card required" subheading, and the footer copyright notice. The fix is straightforward: change `text-gray-400` to `text-gray-500` (approximately 4.6:1 contrast) or `text-gray-600` for even better readability. In the `PipVideoPanel`, `text-white/50` and `text-white/60` on dark backgrounds may also fall below the 3:1 requirement for non-text UI components.

Color-only indicators are used in several places without text alternatives. The invite status badges (Pending amber, Expired gray, Accepted green) rely primarily on background color to communicate status. The `isSpeaking` ring indicator in the video panel uses only a green border with no text or ARIA equivalent. While these elements do contain text labels in some cases, the color coding should not be the sole means of conveying information per WCAG 1.4.1.

5. Section 5: Code Quality & Architecture

Code quality assessment examines the long-term maintainability, scalability, and technical debt of the codebase. A well-architected codebase should have clear separation of concerns, consistent patterns, comprehensive error handling, and minimal duplication. This section identifies architectural anti-patterns, dead code accumulation, naming inconsistencies, and structural issues that will compound in cost as the application grows.

5.1 Architecture Assessment

[HIGH] God File: page.tsx at 1,518 Lines

The main page.tsx file contains both the entire landing page (unauthenticated) and the full dashboard (authenticated) in a single component. This includes auth dialogs, room management, billing panel, agency administration, settings, and invite management. At 1,518 lines, this file violates the single responsibility principle and is extremely difficult to maintain, test, or review. Any change to one feature area risks introducing regressions in others. The file contains multiple complex useEffect hooks with intricate dependency arrays, inline event handlers that should be extracted to custom hooks, and duplicate imports (useAppStore is imported on both line 11 and line 34).

File: src/app/page.tsx

[MEDIUM] No Error Boundaries Anywhere

Zero error boundary components exist in the codebase. A runtime crash in any child component (Whiteboard, Toolbar, VideoPanel) will unmount the entire component tree and display a blank page to the user. For a tutoring platform where users are in active sessions, this is unacceptable. React error boundaries should wrap at least the Whiteboard container, the video panel, and the AI generation components to provide graceful degradation.

File: All component files

[MEDIUM] Duplicate Component Code Across Toolkits

SectionLabel and LockOverlay components are identically copy-pasted across all four toolkit files (MathToolkit, ScienceToolkit, LanguageToolkit, GeneralToolkit). While a shared ToolkitShared.tsx file exists, the toolkits do not import from it. This means any change to these shared components must be replicated four times, creating maintenance burden and inconsistency risk.

File: src/components/toolkits/.tsx*

5.2 Technical Debt Inventory

Issue	Location	Impact
ignoreBuildErrors: true	next.config.ts	Hides all 72 TypeScript errors, including real bugs
reactStrictMode: false	next.config.ts	Disables double-render warnings that catch side effects
noImplicitAny: false	tsconfig.json	Allows implicit any types, weakening type safety
db:push --accept-data-loss	package.json	Dangerous for production; could silently drop columns
images.remotePatterns **	next.config.ts	Wildcard allows images from any external domain
Wildcard Caddyfile XTransformPort	Caddyfile	Potential SSRF vector via port transform
Toast remove delay 1000000ms	use-toast.ts	16-minute cleanup delay causes memory pressure
Duplicate useAppStore import	QuizGenerator.tsx	TS2300 error suppressed by ignoreBuildErrors
Legacy PK migration in register	auth/register/route.ts	Runtime primary key change risk
console.log in 12 files (35+)	Throughout src/	TODO stubs leak implementation details to users

5.3 Dead Code & Unused Dependencies

The codebase contains four npm dependencies that are never imported in any source file. These dependencies increase node_modules size, slow installation, and in the case of next-auth, introduce additional vulnerability surface (uuid CVE). The following table lists all unused dependencies along with their estimated install size and recommended action.

Package	Size	Status	Recommendation
framer-motion	~30 KB gzipped	Not imported anywhere in src/	Remove from package.json
next-auth	~15 KB	Not imported anywhere in src/	Remove from package.json
next-intl	~20 KB	Not imported anywhere in src/	Remove from package.json
next-themes	~3 KB	Only in sonner.tsx (Toaster not used)	Remove with sonner.tsx

Beyond unused dependencies, the codebase contains several categories of dead code within source files. The lib/ai.ts module defines callTextAI and callVisionAI functions that are never called (the API route bypasses them entirely). The lib/geogebra.ts module has executeGeoGebraCommands that only logs to console. Multiple unused imports are scattered across component files: Maximize2 in PipVideoPanel, Plus and LucideIcon in GeoGebraPanel, Highlighter in LanguageToolkit, ChevronLeft/ChevronRight in PageSidebar, and isTutor variable in QuizGenerator. The AI panel "Enable AI" toggle in AIControlPanel fires N individual state updates

synchronously instead of batching them, causing unnecessary re-renders.

Consolidated Remediation Roadmap

This section consolidates all findings from the five audit categories into a prioritized remediation roadmap organized by urgency. Each item includes the affected file, estimated effort, and the specific action required. The roadmap is divided into four phases: Immediate (this week), Short-term (next sprint), Medium-term (next month), and Long-term (quarter). Items are ordered within each phase by the severity of the underlying vulnerability or issue.

Phase 1: Immediate (This Week)

Action	Severity	Category	Effort	Details
Add auth to <code>/api/livekit/token</code>	CRITICAL	Security	2h	Add <code>requireAuth()</code> + room access verification
Add global rate limiting	CRITICAL	Security	4h	Implement <code>@upstash/ratelimit</code> or custom middleware
Move secrets to env-only access	CRITICAL	Security	2h	Remove named exports from <code>lib/*.ts</code> files
Add auth to <code>/api/room/participants</code>	HIGH	Security	1h	Add <code>requireAuth()</code> for GET, rate limit POST
Add skip-to-content link	HIGH	A11y	0.5h	Add as first focusable element in <code>layout.tsx</code>
Wrap landing in main landmark	HIGH	A11y	0.5h	Wrap <code>LandingPage</code> in main element
Fix contrast (text-gray-400)	HIGH	A11y	1h	Change to text-gray-500 across all files
Add aria-label to video controls	HIGH	A11y	1h	All icon buttons in <code>PipVideoPanel</code>

Phase 2: Short-Term (Next Sprint)

Action	Severity	Category	Effort	Details
Add Zod validation to all API routes	HIGH	Security	6h	Library already installed, just needs schemas
Add security headers	HIGH	Security	1h	CSP, X-Frame-Options, HSTS in <code>next.config.ts</code>
Add Prisma indexes	HIGH	Perf	2h	6 composite indexes in <code>schema.prisma</code>
Fix <code>useCredits</code> to use <code>authFetch</code>	HIGH	Func	1h	Replace unauthenticated <code>fetch</code> with <code>authFetch</code>
Fix server-side answer key check	HIGH	Security	3h	Create <code>/api/ai/answer-key</code> endpoint with tutor verification

Action	Severity	Category	Effort	Details
Add keyboard support to PiP panel	HIGH	A11y	4h	onKeyDown handlers, tabIndex, role attributes
Fix clickable divs to buttons	HIGH	A11y	2h	Quick Start items and board rows
Consolidate DB queries	MEDIUM	Perf	3h	Eliminate double-fetch in room, usage, profile routes
Add error boundaries	MEDIUM	Quality	3h	Wrap Whiteboard, VideoPanel, AI components
Add htmlFor to all labels	MEDIUM	A11y	1h	Auth forms and billing panel labels

Phase 3: Medium-Term (Next Month)

Action	Severity	Category	Effort	Details
Fix 72 TypeScript errors	HIGH	Quality	8h	Remove ignoreBuildErrors, fix all TS errors
Remove dead dependencies	MEDIUM	Quality	1h	Uninstall framer-motion, next-auth, next-intl
Split page.tsx into modules	MEDIUM	Quality	12h	Extract LandingPage, Dashboard, BillingPanel, etc.
Add caching to usage endpoint	MEDIUM	Perf	2h	Cache-Control headers or unstable_cache
Replace img with next/image	MEDIUM	Perf	1h	3 raw img tags across the codebase
Fix Stripe billing wrong ID	MEDIUM	Func	2h	Use subscription item ID for createUsageRecord
Connect AI to real Anthropic	HIGH	Func	8h	Wire lib/ai.ts into the API route
Fix KaTeX trust: true XSS	MEDIUM	Security	1h	Set trust: false in lib/katex.ts
Run npm audit fix	MEDIUM	Security	2h	Resolve 25 dependency vulnerabilities
Fix duplicate toolkit components	LOW	Quality	1h	Import from ToolkitShared.tsx

Phase 4: Long-Term (Quarter)

Action	Severity	Category	Effort	Details
Implement real whiteboard (Tldraw)	HIGH	Func	40h	Full Tldraw integration with Yjs persistence
Implement real LiveKit video	HIGH	Func	30h	Real token signing, room connection, media tracks
Create /api/stripe/checkout	HIGH	Func	8h	Server-side checkout session creation

Action	Severity	Category	Effort	Details
Add aria-live to dynamic content	MEDIUM	A11y	4h	Waiting room, toast, copy confirmation
Fix heading hierarchy	MEDIUM	A11y	2h	Consistent h1-h3 structure across all pages
Add request ID tracing	LOW	Quality	4h	Unique request IDs in all error logs
Enable reactStrictMode	LOW	Quality	4h	Fix side effects revealed by strict mode

Appendix A: Full Vulnerability Index

This appendix provides a complete index of all vulnerabilities, issues, and findings discovered during the audit, organized by severity. Each entry includes the finding ID, severity level, affected component, and a brief description. Cross-references to the detailed analysis in the main report sections are provided for each finding.

ID	Severity	Category	Component	Description
V-01	CRITICAL	Security	/api/livekit/token	No authentication on token generation endpoint
V-02	CRITICAL	Security	All API routes	No rate limiting on any endpoint
V-03	CRITICAL	Security	lib/*.ts	Secret keys exported from client-accessible modules
V-04	HIGH	Security	/api/room/participants	No authentication on GET or POST
V-05	HIGH	Security	AnswerKeyModal	Client-side only security gate for answer keys
V-06	HIGH	Security	hocuspocus-server	No room access verification on WebSocket connections
V-07	HIGH	Security	next.config.ts	No CORS or security headers configured
V-08	MEDIUM	Security	/api/agency/invite/[code]	Invite status leakage to unauthenticated users
V-09	MEDIUM	Security	/api/room	Room content exposed to unauthenticated users
V-10	MEDIUM	Security	All API routes	Zod installed but never used for validation
V-11	MEDIUM	Security	lib/katex.ts	trust: true enables HTML injection via LaTeX
V-12	MEDIUM	Security	BrandedHeader, WaitingRoom	Agency logo URL not validated (XSS risk)
V-13	MEDIUM	Security	/api/auth/register	Primary key modification at runtime
V-14	MEDIUM	Security	/api/ai/action	No rate limiting on credit-consuming endpoint
V-15	CRITICAL	Depend	effect (Prisma)	AsyncLocalStorage contamination (npm audit)
V-16	HIGH	Depend	sharp@0.34.3	4 libvips CVEs
V-17	HIGH	Depend	defu	Prototype pollution
V-18	HIGH	Depend	brace-expansion	Zero-step DoS
V-19	HIGH	Depend	@babel/core	Arbitrary file read
V-20	MODERATE	Depend	uuid (next-auth)	Buffer bounds check missing
A-01	HIGH	A11y	WCAG 2.1.1	PiP video panel entirely mouse-only (keyboard inaccessible)
A-02	HIGH	A11y	WCAG 2.1.2	Potential keyboard trap in WaitingRoom overlay
A-03	HIGH	A11y	WCAG 2.4.1	No skip-to-content link

ID	Severity	Category	Component	Description
A-04	HIGH	A11y	WCAG 1.3.1	No main landmark on landing page
A-05	HIGH	A11y	WCAG 4.1.2	Clickable divs lack roles and keyboard support
A-06	HIGH	A11y	WCAG 4.1.2	Video panel buttons lack aria-label
A-07	HIGH	A11y	WCAG 4.1.2	Duplicate nested toolbar roles
A-08	HIGH	A11y	WCAG 1.4.3	text-gray-400 fails 4.5:1 contrast requirement
A-09	HIGH	A11y	WCAG 3.3.2	Labels without htmlFor associations
A-10	HIGH	A11y	WCAG 3.3.1	Error messages lack aria-live regions
P-01	HIGH	Perf	next.config.ts	72 TypeScript errors hidden by ignoreBuildErrors
P-02	HIGH	Perf	Bundle	Prisma client (27 KB) in client bundle
P-03	MEDIUM	Perf	prisma/schema	6 missing database indexes
P-04	MEDIUM	Perf	API routes	Redundant DB queries (3x user lookups)
P-05	MEDIUM	Perf	All API routes	Zero caching infrastructure
Q-01	HIGH	Quality	page.tsx	God file: 1,518 lines, multiple responsibilities
Q-02	MEDIUM	Quality	All components	No error boundaries anywhere
Q-03	MEDIUM	Quality	toolkits/	Duplicate SectionLabel/LockOverlay across 4 files
Q-04	MEDIUM	Quality	12 files	35+ console.log TODO stubs in production code
Q-05	LOW	Quality	package.json	4 unused npm dependencies

Appendix B: Source File Inventory

The following table provides a complete inventory of all source files in the Superboard project, organized by directory. File sizes are approximate line counts as measured during the audit. Files marked with an asterisk (*) contain known issues identified in the main report sections.

File Path	Lines	Description
src/app/layout.tsx	52	Root layout with fonts, metadata, Toaster
src/app/page.tsx *	1,518	Landing page + Dashboard (god file)
src/app/room/[roomId]/page.tsx *	170	Whiteboard room page
src/app/invite/[code]/page.tsx	343	Invite acceptance page
src/app/globals.css *	~200	Design system CSS
src/app/api/route.ts	4	Health check
src/app/api/room/route.ts *	196	Room CRUD
src/app/api/room/templates/route.ts	113	Template management
src/app/api/room/participants/route.ts *	113	Participant join/list (no auth)
src/app/api/auth/register/route.ts *	93	User registration
src/app/api/auth/profile/route.ts *	85	User profile
src/app/api/ai/action/route.ts *	131	AI proxy (always placeholder)
src/app/api/stripe/webhook/route.ts	249	Stripe webhook handler
src/app/api/livekit/token/route.ts *	110	LiveKit token (no auth, mock)
src/app/api/agency/invite/route.ts	175	Agency invite management
src/app/api/agency/invite/[code]/route.ts *	176	Invite view/accept
src/app/api/agency/invite/[code]/cancel/route.ts	85	Cancel invite
src/app/api/agency/students/route.ts *	86	Student count for billing
src/app/api/agency/subtutors/[tutorId]/route.ts	89	Remove sub-tutor
src/app/api/usage/current/route.ts *	85	Current usage stats
src/app/api/usage/fingerprint/route.ts *	76	Anti-fraud fingerprint
src/app/api/usage/agency/route.ts	114	Agency aggregate usage
src/components/canvas/Whiteboard.tsx *	280	Main whiteboard (placeholder)
src/components/canvas/Toolbar.tsx	219	Dynamic toolbar
src/components/canvas/PageSidebar.tsx *	84	Multi-page sidebar
src/components/canvas/ImageCompressor.ts	128	Image compression utility
src/components/ai/AIControlPanel.tsx	292	AI feature toggle panel
src/components/ai/QuizGenerator.tsx *	276	Quiz generation form

File Path	Lines	Description
src/components/ai/WorksheetGenerator.tsx *	352	Worksheet generation form
src/components/ai/AnswerKeyModal.tsx *	215	Answer key viewer
src/components/ai/GeoGebraPanel.tsx *	574	GeoGebra graphing (placeholder)
src/components/ai/LazyGeoGebraPanel.tsx	22	Lazy import wrapper
src/components/premium/PaywallModal.tsx *	170	Upgrade modal
src/components/premium/UsageBar.tsx *	101	Fixed bottom usage bar
src/components/video/PipVideoPanel.tsx *	547	Floating video panel (mock)
src/components/video/RecordButton.tsx *	175	Recording toggle (disabled)
src/components/student/WaitingRoom.tsx *	191	Student waiting overlay
src/components/student/NameEntryModal.tsx	136	Student name entry
src/components/branding/BrandedHeader.tsx *	69	Agency branded header
src/components/branding/BrandedPdfExport.tsx *	143	PDF export (non-functional)
src/components/toolkits/MathToolkit.tsx *	230	Math subject tools
src/components/toolkits/ScienceToolkit.tsx *	204	Science subject tools
src/components/toolkits/LanguageToolkit.tsx *	237	Language subject tools
src/components/toolkits/GeneralToolkit.tsx *	172	General tools
src/components/toolkits/ToolkitShared.tsx	24	Shared toolkit components
src/hooks/useCredits.ts *	79	Credit usage polling (broken)
src/hooks/useTheme.ts	115	Dynamic CSS theming
src/hooks/useFocusMode.ts *	70	Viewport sync (stub)
src/hooks/use-mobile.ts *	19	Mobile breakpoint detection
src/hooks/use-toast.ts *	193	Toast notification system
src/lib/supabase.ts	49	Supabase client config
src/lib/db.ts	31	Prisma client singleton
src/lib/auth.ts	74	JWT verification
src/lib/auth-fetch.ts *	66	Authenticated fetch wrapper
src/lib/stripe.ts *	103	Stripe client (secrets exported)
src/lib/stripe-billing.ts *	344	Metered billing (wrong ID)
src/lib/usage.ts	153	Tier limit checking
src/lib/ai.ts *	132	Anthropic client (dead code)
src/lib/fingerprint.ts	52	FingerprintJS client
src/lib/livekit.ts *	71	LiveKit config (secrets exported)

File Path	Lines	Description
src/lib/mathpix.ts *	69	Mathpix OCR (secrets exported)
src/lib/katex.ts *	65	KaTeX rendering (trust: true)
src/lib/geogebra.ts *	114	GeoGebra commands (console.log)
src/lib/utlis.ts	6	cn() utility
src/store/app-store.ts *	170	Zustand global state
src/types/index.ts	172	Shared type definitions
src/middleware.ts *	54	Custom domain routing (commented out)
prisma/schema.prisma *	187	Database schema (no indexes)